Compilers Project

SafeC

A Memory-Safe C-Family Language on LLVM

Team Members

Atul Boyal	CS24BTECH11008
Akshat Banzal	CS24BTECH11005
Aric Maji	CS24BTECH11007
Rishika Shreya Fadi	CS24BTECH11052
Harikrishna S	CS24BTECH11030
Chakram Siri	CS24BTECH11017
Aare Renu Shri	CS24BTECH11001

ABSTRACT

SafeC is a systems programming language that aims to provide memory safety while keeping the syntax and low-level programming style of C. It is designed to prevent common memory-related errors such as out-of-bounds access, null pointer dereferencing, uninitialized memory reads, use-after-free, double-free, and memory leaks in safe code. SafeC uses features such as non-null pointers by default, nullable pointers, ownership, and borrowing to achieve these safety guarantees. SafeC does not use a garbage collector or require a large runtime. Instead, the compiler performs safety checks before generating LLVM IR. It performs static semantic and ownership verification directly over the Abstract Syntax Tree (AST) before lowering to LLVM IR to perform checks related to initialization, null pointers, ownership, and borrowing. After these checks, the program is converted to LLVM IR, which can then be optimized and compiled for the target machine. SafeC also follows the C ABI, so it can work with existing C libraries and object files. This makes it possible to introduce SafeC into existing C projects without rewriting the entire project.

Index

1 Introduction **3**

2 Domain **4**

3 SafeC Language Specification **4**

3.1 Design Philosophy & Core Guarantees 4

3.2 The Pointer Taxonomy & Null Safety 6

3.3 Ownership and Move Semantics (`own`) 7

3.4 Borrowing and Lexical Lifetimes (`T*` and `mut T*`) 8

3.5 Memory Lifecycle Management (`new`, `free`, and `defer`) 9

3.6 The Escape Hatch: `trusted` and `raw` Pointers 9

4 Unique Selling Points (USP) **10**

4.1 Rust-like memory safety, but with a simpler model 10

4.2 Borrows are second-class, so checking stays local 11

4.3 C-like syntax and programming model, but not C's semantics 11

4.4 Safety is checked at compile time 11

4.5 No garbage collector, no new runtime 12

4.6 Incremental adoption 12

4.7 The central idea 12

5 Comparison with Existing Languages and Memory-Safety Approaches . . . **12**

5.1 SafeC — Proposed System 12

5.2 ISO C 13

5.3 Rust 13

5.4 Checked C 14

5.5 Hylo 14

6 Complete Annotated Case Study: Singly Linked List **15**

6.1 Key Verification Properties in the Case Study 16

7 Implementation Overview **17**

7.1 Compilation Flow 17

7.2 C Library Integration 18

1 Introduction

Memory-safety bugs are a major source of security problems in systems software. Systems software such as operating systems, embedded applications, databases, and networking software often uses C because it provides low-level control over memory and good performance. However, this control also makes it easy to introduce memory-related errors. C does not automatically check whether a pointer is valid or whether an array access is within its bounds. As a result, problems such as out-of-bounds access, null pointer dereferencing, uninitialized memory reads, use-after-free, and double-free can occur in C programs.

There are different approaches to solving these problems. Rust provides memory safety through ownership and borrowing, but it has a different programming model from C and can require changes when working with existing C code. Garbage-collected languages handle memory automatically, but require a runtime and may not be suitable for all systems applications.

SafeC targets this gap by combining the familiar programming style of C with stronger memory-safety guarantees. The language is designed so that common memory errors can be prevented in safe code while still allowing programmers to work with low-level memory and existing C software. SafeC also supports incremental adoption, allowing SafeC components to work alongside existing C code instead of requiring an entire project to be rewritten.

Key advantages of SafeC include:

Memory Safety:
Prevents common errors such as out-of-bounds access, null pointer dereferencing, uninitialized reads, use-after-free, and double-free errors in safe code.

Familiar Syntax:
Retains the C-style syntax, primitive types, structures, functions, and low-level programming model so that programmers familiar with C can adapt more easily.

Ownership and Borrowing:
Uses unique ownership and restricted borrowing to keep track of dynamically allocated memory and prevent invalid memory accesses.

No Garbage Collector:
SafeC does not require a garbage collector or a managed runtime, making it suitable for systems where direct and predictable memory management is important.

C Interoperability:
Follows the platform C ABI and supports interaction with existing C libraries and object files, allowing SafeC code to be introduced into existing C projects.

Controlled Unsafe Code:
Operations that cannot be verified by the compiler can still be written using explicit trusted blocks (using raw pointers), making unsafe parts of the program easier to identify and review.

SafeC is intended for systems where low-level control and memory safety are both important. It can be used alongside existing C code instead of requiring an entire project to be rewritten. The compiler performs static semantic and ownership verification directly over the Abstract Syntax Tree (AST) and then converts the checked program into LLVM IR.

2 Domain

SafeC is mainly intended for systems programming applications where programmers need direct control over memory but also want to avoid common memory-safety problems. C is widely used in such applications because of its performance, low-level control, and large collection of existing libraries. SafeC is designed to work in this environment rather than replacing it completely. It can be used in situations where manual memory management is required, but errors such as invalid pointer access, use-after-free, and out-of-bounds access need to be reduced.

Some of the areas where SafeC can be useful are:

Operating Systems:
System and kernel-level code makes extensive use of pointers and manually managed memory. SafeC provides ownership and borrowing rules while still allowing low-level operations through unsafe code when required.

Embedded Systems:
Embedded programs often have limited memory and performance requirements. Since SafeC does not depend on a garbage collector or a large runtime system, it can be considered for applications where predictable memory management is important.

Networking Software:
Network programs frequently work with buffers and blocks of memory whose sizes need to be tracked carefully.

Databases and Storage Systems:
These applications often involve manual memory management and existing C libraries. SafeC's support for the C ABI allows it to work with existing C object files and libraries instead of requiring the complete system to be rewritten.

Compilers and Language Runtimes:
Compilers and runtime systems require both low-level control and careful memory management. SafeC's ownership and borrowing features can help manage dynamically allocated data while still allowing interaction with low-level code.

A major focus of SafeC is therefore working within the existing C ecosystem. SafeC programs can interact with C code and libraries, while parts that cannot be checked automatically can be placed inside explicit trusted blocks (using raw pointers). This makes SafeC suitable for gradually introducing memory-safe code into existing systems instead of requiring an entire C codebase to be rewritten at once.

3 SafeC Language Specification

3.1 Design Philosophy & Core Guarantees

3.1.1 The SafeC Thesis

In traditional C, a single pointer type—`T*`—is overloaded to express every possible memory relationship. A `T*` may represent sole ownership of a heap allocation, a temporary alias for reading a structure, a mutable out-parameter, an iterator stepping through an array, or a null sentinel. Because the type system makes no distinction among these roles, the C compiler cannot prevent a programmer from freeing a borrowed pointer, mutating data through an alias while

another pointer reads it, dereferencing a null value, or accessing heap memory after its owner has released it.

SafeC eliminates these vulnerabilities without introducing garbage collection, reference counting, or hidden runtime metadata. It achieves this by extending a strict subset of C with a single organizing discipline: an ownership-based memory model checked entirely at compile time. Every pointer in SafeC explicitly encodes its ownership status, its mutability rights, and its nullability in its type signature. Anything that compiles outside of an explicitly marked trusted block is statically guaranteed to be free of use-after-free errors, double frees, null-pointer dereferences, and uncontrolled pointer aliasing.

3.1.2 The C Baseline

SafeC is designed from the ground up as a minimal, predictable systems language. Rather than inheriting the entirety of ISO C and attempting to retrofit safety onto legacy constructs, SafeC starts from a clean foundational subset of C and adds only the constructs required to express and verify memory ownership. C keywords and features not listed in this specification—such as unchecked union types, implicit pointer conversions, or ambient pointer arithmetic—are omitted by design and introduced only as the formal ownership model is extended to govern them.

3.1.3 Lexical Summary

SafeC retains standard C primitive types, structured control flow, and aggregate struct definitions, while introducing seven keywords dedicated to memory safety and three literal values for boolean and null representation.

Inherited from C:

Category	Keywords
Primitive Types	`void`, `bool`, `char`, `int`, `float`
Control Flow	`if`, `else`, `switch`, `case`, `default`, `for`, `while`, `break`, `continue`, `return`
User-Defined Types	`struct`
Literals	`true`, `false`

New in SafeC:

Construct	Role
`own`	Marks a pointer as the sole, exclusive owner of the heap memory it references
`mut`	Marks a borrowed pointer as writable (read-write access)
`raw`	Marks an unchecked pointer exempt from static ownership and borrowing rules
`trusted`	Delimits a lexical block in which the programmer assumes responsibility for memory safety
`new`	Allocates typed heap storage and yields a non-null `own` pointer to it
`free`	Consumes an `own` pointer and releases its underlying heap allocation
`defer`	Schedules a statement to execute automatically when the enclosing `{}` block exits
`null`	The null pointer literal, assignable exclusively to nullable pointer types (`T*?`)

3.2 The Pointer Taxonomy & Null Safety

3.2.1 Unified Pointer Classification

Every safe pointer in SafeC is classified along two orthogonal axes:

- **Capability** (`own T*`, `T*`, `mut T*`): Governs who is responsible for freeing the underlying memory and whether the pointee may be mutated.
- **Nullability** (`T*` vs. `T*?`): Governs whether the pointer is statically guaranteed to reference a valid object or may hold the sentinel value `null`.

An unchecked pointer form, `raw T*`, exists outside this safe taxonomy for low-level systems operations (Section 3.6).

Pointer Form	Classification	Can Read?	Can Write?	Owns Memory?
`own T*`	Owning Pointer	Yes	Yes	Yes (sole owner)
`T*`	Read Borrow	Yes	No	No
`mut T*`	Write Borrow	Yes	Yes	No
`raw T*`	Raw Pointer	In trusted only	In trusted only	No (unchecked)

3.2.2 Non-Nullable by Default (`T*`)

In C, every pointer is implicitly nullable, forcing programmers to either litter code with defensive `!= NULL` checks or risk catastrophic segmentation faults. SafeC reverses this default: an unadorned pointer (`own T*`, `T*`, or `mut T*`) is statically non-nullable. It is a compile-time type error to assign `null` to a non-nullable pointer. Consequently, any function accepting a `T*`, `mut T*`, or `own T*` may dereference it immediately with zero runtime null checks.

3.2.3 Nullable Pointers and Flow-Sensitive Narrowing (`T*?`)

When the absence of a value is part of a program's domain logic—such as the tail pointer of a linked list or an optional return handle—SafeC provides the nullable pointer suffix `?` (`own T*?`, `T*?`, `mut T*?`).

A nullable pointer `T*?` cannot be dereferenced (`*p` or `p->field`), passed to a function expecting a non-nullable `T*`, or freed via `free(p)` directly. To operate on the underlying value, the program must first prove at that point in execution that the pointer is non-null using an explicit comparison against `null`:

```
void inspect_node(Node*? maybe_node) {
    // int x = maybe_node->value; // Compile-time error: maybe_node may be null
    if (maybe_node != null) {
        int x = maybe_node->value; // Valid: narrowed to non-nullable Node* inside
    branch
    }
}
```

Within the body of an `if (p != null)` branch or a `while (p != null)` loop, the compiler narrows the type of `p` from `T*?` to `T*`, permitting safe dereferencing, borrowing, or moving into a non-nullable `own T*` binding.

3.3 Ownership and Move Semantics (`own`)

3.3.1 The Single-Owner Invariant

Heap memory in SafeC is governed by a strict single-owner rule: every allocation created by `new` has exactly one `own` pointer referencing it at any given moment. Because ownership is unique, there is never ambiguity about which component of a program is responsible for releasing an object.

3.3.2 Move Semantics and Static Invalidation

Assigning an `own` pointer to another `own` variable, storing it into a struct field, or passing it to a function accepting an `own` parameter performs a move. Moving transfers exclusive ownership of the allocation to the destination and statically invalidates the source binding:

```
own int* a = new int;
*a = 42;

own int* b = a; // Ownership moves from a to b; a is invalidated
// int v = *a;  // Compile-time error: use of moved pointer 'a'
free(b);        // Valid: b is the active owner
```

Any attempt to read, write, borrow, move, or free a variable after its ownership has been moved is rejected at compile time. This single rule eliminates both use-after-free and double-free bugs across the entire language.

3.3.3 Ownership Inside Structs (`struct`)

Structures may contain `own` pointer fields (commonly `own T*?` for dynamic containers and recursive graphs). A struct that contains one or more `own` fields transitively owns the heap allocations reachable through those fields.

To preserve the single-owner invariant without requiring `own` qualifiers on stack-allocated value types, a struct containing an `own` pointer field cannot be copied by value once initialized. Functions that inspect or mutate such a struct must accept it via a read borrow (`StructType*`) or write borrow (`mut StructType*`). When moving an `own` field out of a struct via a write borrow, the programmer must overwrite that field with either a new valid owner or `null` before the borrow ends, ensuring the struct is never left with a dangling or uninitialized `own` handle.

3.4 Borrowing and Lexical Lifetimes (`T*` and `mut T*`)

3.4.1 Read Borrows (`T*`) and Write Borrows (`mut T*`)

Most functions do not need to take ownership of data or deallocate it; they merely need temporary access to read or modify it. SafeC expresses temporary access through borrows:

- **Read Borrow (`T*`):** Created implicitly from an existing pointer or explicitly via the address-of operator `&x`. A read borrow grants immutable, read-only access to the target value. Neither the target nor any `own` pointers nested inside it may be mutated or moved through a `T*`.
- **Write Borrow (`mut T*`):** Created via the mutable address-of operator `&mut x` or derived from an existing `mut` or `own` pointer. A write borrow grants full read-write access to mutate the pointee's fields or swap ownership of its internal `own` pointers, without transferring ownership of the borrowed container itself.

3.4.2 Aliasing and Exclusivity Rules

To prevent mutation hazards (such as modifying a collection while iterating over it), SafeC enforces strict aliasing rules between owners and borrows:

- **Shared Reads or Exclusive Write:** At any point in a block, an object may have one or more active read borrows (`T*`), or a single active write borrow (`mut T*`), never both simultaneously.
- **Owner Freezing:** While any borrow (`T*` or `mut T*`) of an object is active, the original owner cannot be freed or moved.

3.4.3 Lexical Borrow Scoping

In SafeC v1, the lifetime of a borrow is strictly bound to the lexical `{}` block in which the borrow binding is declared—no more and no less. A borrow introduced inside a function call expression (such as `list_push(&mut list, 1);`) is scoped to that statement, whereas a borrow assigned to a local variable (`Node*? cur = list->head;`) remains active until the closing `}` of its enclosing block. Future revisions (v2) may relax this rule via control-flow liveness analysis to terminate borrows after their last syntactic use, without altering the underlying safety guarantees.

3.5 Memory Lifecycle Management (`new`, `free`, and `defer`)

3.5.1 Typed Allocation (`new`) and Consumption (`free`)

SafeC replaces C's untyped `malloc(sizeof(T))` and `void*` casts with the type-safe `new` operator:

- **`new T`:** Allocates sufficient heap memory to store a value of type `T` and returns a non-nullable owning pointer of type `own T*`.
- **`free(p)`:** Accepts a non-nullable `own T*` pointer `p`, releases the underlying heap memory, and consumes `p`. After `free(p)`, `p` is statically invalidated exactly as if it had been moved. Calling `free` on a read borrow (`T*`), a write borrow (`mut T*`), a nullable pointer (`own T*?` that has not been narrowed), or an already-moved pointer is a compile-time error.

3.5.2 Scope-Bound Cleanup (`defer`)

In C functions with multiple conditional return paths, programmers frequently introduce memory leaks by forgetting to free local allocations before an early exit. SafeC solves this structurally with the `defer` keyword.

`defer <statement>;` schedules its statement to execute automatically when control exits the enclosing `{}` block—whether by reaching the closing brace, executing a `return`, or jumping via `break` or `continue`:

```
{
    own int* x = new int;
    defer free(x);
    *x = 100;
    // ... complex control flow using x ...
} // free(x) executes automatically here
```

By placing `defer free(x);` immediately after `own T* x = new T;`, resource acquisition and release are paired at the top of the scope, eliminating leak paths across early returns.

3.5.3 Teardown of Recursive Structures

When freeing a struct `n` that itself contains an `own` field (such as `own Node*? next` in a linked list node), SafeC prevents accidental memory leaks of transitive allocations. Before calling `free(n)`, any child `own` pointer inside `n` must first be moved out to another owner and cleared to null (`n->next = null;`), proving to the compiler that downstream heap nodes have been accounted for before the parent node is destroyed.

3.6 The Escape Hatch: `trusted` and `raw` Pointers

3.6.1 The Pragmatic Boundary

A systems programming language must occasionally interface with hardware registers, operating system kernels, custom memory allocators, or legacy C libraries—domains where memory layout and aliasing invariants cannot be proven solely through static ownership types. Rather than rejecting these workloads or weakening checking across the entire program, SafeC isolates unverifiable operations behind an explicit lexical boundary: the `trusted` block.

3.6.2 Unchecked Pointers (`raw T*`)

To operate on memory outside the ownership model, SafeC provides the `raw T*` pointer type:

- **Unrestricted Creation:** A `raw T*` variable may be declared, initialized, copied, or passed around anywhere in safe code with zero ownership or borrow-checking obligations. Creating a raw pointer never invalidates an owner or freezes a borrow.
- **Restricted Dereferencing:** A `raw T*` cannot be dereferenced (`*p`, `p->field`) or subjected to pointer arithmetic in standard SafeC code. Those operations are permitted exclusively inside a `trusted` block.

3.6.3 The `trusted { ... }` Contract

Writing `trusted { ... }` suspends static ownership, borrow, and raw-pointer dereference checks for the statements inside the block:

```
void write_mmio_register(raw int* reg_addr, int command) {
    trusted {
        *reg_addr = command; // Permitted: programmer vouches for address validity
    }
}
```

In writing `trusted`, the programmer enters an explicit contract with the compiler: the code inside the block must manually uphold SafeC's invariants (not fabricating invalid `own` pointers or corrupting active borrows) so that callers of the enclosing function continue to enjoy full memory safety. When auditing a SafeC codebase for memory corruption, reviewers need only inspect `trusted` blocks—a fraction of the total program text.

4 Unique Selling Points (USP)

Most approaches to making C memory-safe require changing something about how C programmers normally work. Rust provides strong memory safety, but uses a different programming model with concepts such as ownership and lifetimes. Garbage-collected languages manage memory automatically, but need a runtime. Static analyzers and sanitizers can find memory errors, but usually detect them rather than preventing them during compilation.

SafeC aims to provide another option: memory safety for common memory errors while keeping C-like syntax, a C-compatible ABI, and direct control over memory.

Concretely, this comes down to five things:

4.1 Rust-like memory safety, but with a simpler model

SafeC uses ownership and borrowing rules to catch important memory errors such as use-after-free, double-free, and leaks of uniquely-owned memory at compile time.

For example, when ownership of a pointer is moved to another variable, the original variable can no longer be used to access or free that memory. This allows the compiler to detect many memory-management errors before the program runs.

SafeC does not try to reproduce all of Rust's safety models. Its borrowing rules are intentionally

more restricted. The goal is to give strong memory-safety guarantees while using a smaller and easier-to-learn set of rules.

4.2 Borrows are second-class, so checking stays local

A SafeC borrow is temporary and cannot be stored in a struct, global variable, or heap object. t can exist only as a local variable or function parameter scoped to a lexical block.

For example, a function can temporarily borrow some memory:

```
void print_value(int* x) {
    printf("%d", *x);
}
```

but the borrowed reference cannot be stored somewhere where it could outlive the function.

This restriction is important because it makes the borrow checker intraprocedural. Since borrows cannot escape into stored data, the compiler can check borrowing rules one function at a time instead of tracking references across the entire program.

This is also why SafeC does not need explicit lifetime parameters like Rust. SafeC gives up some flexibility in where references can be stored in exchange for a much simpler borrowing system.

4.3 C-like syntax and programming model, but not C's semantics

SafeC is designed to look familiar to C programmers. Control flow, structs, functions, and basic programming constructs remain C-like.

However, SafeC changes some important C semantics to make the language safer.

For example, pointer types can carry safety information:

```
int*       // non-null safe pointer
int*?      // nullable pointer
own int*   // owning pointer
raw int*   // raw/unsafe pointer
```

SafeC also checks that variables are initialized before they are used, removes unsafe implicit pointer conversions, and gives defined behavior to operations where the language specification requires it.

So safety is built into the language's type system and semantics.

4.4 Safety is checked at compile time

SafeC performs important safety checks during compilation rather than relying on runtime mechanisms.

Ownership tracking, borrow checking, and definite-initialization checking are handled by the compiler before the program is generated. For example, the compiler can detect when ownership has already been moved, when a borrow would outlive its allowed scope, or when a variable is used before being initialized.

Because these checks are performed by the compiler, they do not require runtime ownership tracking or a garbage collector.

This allows SafeC to provide strong memory-safety guarantees while keeping the generated program close to the low-level execution model of C.

4.5 No garbage collector, no new runtime

SafeC does not use a garbage collector or a managed heap.

An owned pointer is still represented at the machine level as an ordinary pointer, and memory can be allocated using the platform's normal allocation mechanisms such as `malloc`.

The compiler handles ownership and borrowing at compile time rather than relying on a garbage collector during program execution.

This keeps SafeC suitable for systems programming where programmers may need predictable execution and direct control over memory.

4.6 Incremental adoption

SafeC is designed to work alongside existing C code instead of requiring an entire codebase to be rewritten.

SafeC code is compiled through LLVM into native object code and uses the standard C ABI. This allows SafeC object files to be linked with existing C object files and libraries using the normal system linker.

Therefore, an existing C project can gradually replace individual modules with SafeC instead of rewriting the entire project at once.

4.7 The central idea

The main technical idea behind SafeC is second-class borrows.

By preventing borrowed references from being stored in long-lived data structures, SafeC limits how far a borrow can escape. This allows the compiler to perform borrow checking locally, without requiring explicit lifetime annotations.

In other words:

> *Restrict where borrows can go → simplify lifetime reasoning → avoid lifetime annotations → make memory safety easier to use while keeping a C-like programming model.*

5 Comparison with Existing Languages and Memory-Safety Approaches

5.1 SafeC — Proposed System

Abstraction Level	C-family systems programming language that retains C-like expressions, statements, primitive types, structs, and functions while introducing safer pointer and array types.
Language Integration	Designed to work alongside existing C source files rather than being a C-source-compatible superset.
Memory Safety	Provides compile-time ownership, borrowing, null-safety, and definite-initialization checks.
Runtime Overhead	Ownership, borrowing, null safety, and initialization are intended to have no runtime cost; bounds and arithmetic checks may remain at runtime and can potentially be optimized away by LLVM.
Borrowing Model	Uses second-class borrows that cannot be stored in globals, heap objects, or ordinary struct fields, avoiding explicit lifetime parameters.
C Interoperability	Uses the platform C ABI, supports linking with C objects/libraries, and can export C-callable functions.
Debugging Support	Targets LLVM IR and therefore intends to support GDB, LLDB, perf, Instruments, sanitizers, and LLVM-based LTO workflows.
Concurrency	Data-race freedom is explicitly outside the scope of SafeC v1.
Performance Goal	The paper proposes a target of being within 10% of equivalent C at `-O2`, but this is a future target to be measured, not an established result.
Target Use Cases	Intended for memory-safe systems software requiring C-like programming, predictable performance, LLVM tooling, and direct interoperability with existing C code.

5.2 ISO C

Abstraction Level	Low-level systems programming language with manual memory management and unrestricted pointer operations.
Language Integration	Native C language; provides the baseline C programming and ABI model that SafeC is designed to interoperate with.
Memory Safety	Does not provide compile-time guarantees against common memory errors such as out-of-bounds access, use-after-free, or dangling pointers.
Runtime Overhead	Does not inherently impose SafeC-style bounds, ownership, or arithmetic-overflow checks.
Debugging Support	Mature compiler and debugging ecosystem; SafeC intends to retain compatibility with LLVM-based debugging and profiling tools.
Interoperability	Native C interoperability; SafeC object files use the platform C ABI and can link directly with C objects and libraries.
Target Use Cases	Operating systems, embedded systems, networking, high-performance software, and other low-level systems applications.

Comparison with SafeC: ISO C treats every `T*` as an unrestricted, potentially null address and leaves `free` calls unchecked, whereas SafeC partitions pointers into owning (`own T*`), read/write borrowed (`T*` and `mut T*`), and explicitly nullable (`T*?`) forms. SafeC preserves C's explicit heap lifecycle via `new`, consuming `free`, and `defer`, while isolating unchecked `raw T*` dereferences and pointer arithmetic strictly inside `trusted` blocks.

5.3 Rust

Abstraction Level	Systems programming language based on ownership, borrowing, lifetimes, and a strong static type system.
Language Integration	Standalone language with explicit C foreign-function interfaces through `extern "C"`.
Memory Safety	Provides compile-time memory-safety guarantees through ownership and borrowing without requiring a garbage collector.
Runtime Overhead	Ownership and borrowing are primarily compile-time mechanisms; Rust does not require a garbage collector for memory management.
Concurrency	Provides type-system mechanisms such as `Send` and `Sync` for expressing thread-safety properties.
Interoperability	Supports C interoperability through its foreign-function interface and C ABI.
Target Use Cases	Systems programming, embedded software, performance-sensitive applications, networking, and applications requiring strong compile-time safety.

Comparison with SafeC: Both languages enforce compile-time move semantics, exclusive mutable borrowing, non-null defaults, and explicit escape hatches (`unsafe` vs. `trusted` with `raw T*`). However, SafeC avoids Rust's explicit lifetime annotations (`'a`), non-lexical borrow analysis, and implicit RAII destructors (`Drop`) by tying v1 borrow scopes strictly to lexical `{}` blocks and pairing explicit `new`/`free` deallocation with `defer`.

5.4 Checked C

Abstraction Level	Extension of C that adds checked pointer types and bounds annotations while retaining the C programming model.
Language Integration	Designed to allow existing C code to be used and incrementally converted to Checked C.
Memory Safety	Adds spatial memory-safety mechanisms such as bounds checking and checked pointer types.
Runtime Overhead	Bounds checks can be inserted when the compiler cannot establish safety statically.
Temporal Safety	The SafeC paper distinguishes Checked C from SafeC by describing Checked C's primary model as bounds/spatial checking rather than SafeC's ownership-based temporal safety.
Interoperability	Maintains close compatibility with existing C code, supporting incremental adoption.
Target Use Cases	Existing C codebases where memory-safety checking can be introduced incrementally.

Comparison with SafeC: Checked C extends C pointers primarily for spatial bounds checking and non-null guarantees while leaving deallocation lifetimes under unchecked manual control. SafeC instead centers its type system on temporal and aliasing safety: moving an `own T*` or passing it to `free(x)` statically invalidates `x` thereafter, eliminating use-after-free and double-free bugs at compile time.

5.5 Hylo

Abstraction Level	Systems programming language based on mutable value semantics, projections, and scoped access rather than conventional reference types.
Language Integration	Uses its own language model rather than attempting to remain syntactically equivalent to C.
Memory Safety	Uses mutable value semantics and scoped access mechanisms to enforce its memory-safety model while targeting systems-level performance.
Borrowing Model	Its second-class reference concepts are particularly relevant to SafeC; the SafeC paper identifies Hylo as the source of inspiration for SafeC's second-class borrow idea.
Runtime Overhead	Designed for ahead-of-time native compilation with mutable value semantics intended to avoid unnecessary copies and hidden allocation costs.
Target Use Cases	Systems programming using mutable value semantics and strong control over references and memory.

Comparison with SafeC: Hylo inspires SafeC's approach to lifetime-free, lexically scoped borrowing, demonstrating that temporal safety can be checked without Rust-style lifetime parameters. However, Hylo replaces pointers with mutable value semantics and projections, whereas SafeC retains C's explicit pointer indirection (`own T*`, `T*`, `mut T*`), heap management (`new`, `free`, `defer`), and `raw T*` escape hatches inside `trusted` blocks.

6 Complete Annotated Case Study: Singly Linked List

The following complete program demonstrates how SafeC's ownership, borrowing, null-narrowing, and lifecycle rules work together in a classic pointer-heavy data structure: a dynamic singly linked list.

```
struct Node {
    int value;
    own Node*? next;    // Owns the remainder of the list, or null at the tail
};

struct List {
    own Node*? head;    // Owns the first node, transitively owning the entire chain
};

List list_new() {
    return (List){ .head = null };
}

// Mutates the list's head pointer, requiring a write borrow (mut List*).
void list_push(mut List* list, int value) {
    own Node* n = new Node;
    n->value    = value;
    n->next     = list->head;   // Move: previous head chain now belongs to n->next
    list->head  = n;            // Move: ownership of n transfers into list->head
}

// Mutates both the list and the caller's output integer via write borrows.
bool list_pop(mut List* list, mut int* out_value) {
    if (list->head == null) return false;
```

```
    own Node* n = list->head;    // Move front node out (narrowed to non-null own
    Node*)
    list->head  = n->next;       // Move remaining chain back into list->head
    n->next     = null;          // Detach downstream ownership before freeing n
    *out_value  = n->value;      // Write result through mutable borrow out_value
    free(n);                     // Consume and deallocate the detached node n
    return true;
}

// Read-only traversal: takes an immutable read borrow (List*) and never alters
    ownership.
void list_print(List* list) {
    Node*? cur = list->head;     // Nullable read borrow of the head node
    while (cur != null) {        // Narrows cur to non-nullable Node* inside loop
    body
        printf("%d ", cur->value);
        cur = cur->next;         // Advance read borrow to the next node
    }
    printf("\n");
}

// Releases all heap nodes by walking the chain through a write borrow.
void list_free(mut List* list) {
    own Node*? cur = list->head;
    list->head     = null;       // Leave the borrowed List in a valid empty state
    while (cur != null) {
        own Node* n = cur;       // Move narrowed non-null node out of cur
        cur         = n->next;   // Move remainder of the chain into cur
        n->next     = null;      // Clear child owner before deallocation
        free(n);                 // Safely release current node
    }
}

int main() {
    List list = list_new();
    list_push(&mut list, 1);
    list_push(&mut list, 2);
    list_push(&mut list, 3);

    list_print(&list);               // Outputs: 3 2 1

    int x;
    list_pop(&mut list, &mut x);
    printf("popped: %d\n", x);       // Outputs: popped: 3
    list_print(&list);               // Outputs: 2 1

    list_free(&mut list);
    return 0;
}
```

6.1 Key Verification Properties in the Case Study

Zero Null Dereferences:

Both `list->head` and `Node::next` are typed as `own Node*?`. Neither `list_pop`, `list_print`,

nor `list_free` can dereference or free a node without first passing an `if (list->head == null)` or `while (cur != null)` check, which narrows the pointer to a non-nullable `Node*` or `own Node*`.

Leak-Free Node Splicing:
In `list_pop` and `list_free`, the compiler tracks the ownership of `n->next`. By moving `n->next` back into `list->head` (or `cur`) and setting `n->next = null` prior to `free(n)`, the programmer proves that freeing `n` does not orphan the rest of the list.

Compile-Time Aliasing Protection:
While `list_print(&list)` holds a read borrow of `list` for the duration of the call, no mutation can occur. Conversely, `list_push`, `list_pop`, and `list_free` explicitly document at the call site (`&mut list`) that the list's internal state is being modified.

7 Implementation Overview

7.1 Compilation Flow

The SafeC compiler translates source programs into native executables by following five high-level phases:

1. **Lexical Analysis:** The source code is tokenized by the lexer into a stream of tokens representing C baseline constructs, SafeC keywords (`own`, `mut`, `raw`, `trusted`, `new`, `free`, `defer`), identifiers, literals, and operators. Whitespace and comments are stripped before passing tokens to the parser. Example tokens:

Lexeme		Token
`own`	→	`OWN_KEYWORD`
`mut`	→	`MUT_KEYWORD`
`trusted`	→	`TRUSTED_KEYWORD`
`?`	→	`NULLABLE_PTR`
`list_push`	→	`IDENTIFIER`
`;`	→	`SEMICOLON`

2. **Parsing:** The token stream is parsed into an Abstract Syntax Tree (AST). The parser validates syntax and attaches pointer qualifiers (`own`, `mut`, `raw`, and `?`) to declarations, struct fields, and function parameters, while constructing AST nodes for `trusted` blocks, `defer` statements, and memory operations (`new` and `free`).

3. **Semantic & Ownership Analysis:** The compiler performs type checking and static ownership verification over the AST to ensure that:

   - Non-nullable pointers (`T*`) are never assigned `null`, and nullable pointers (`T*?`) are checked against `null` before use.
   - Ownership transfers (`own`) invalidate the source variable, preventing use-after-free and double-free errors.
   - Read (`T*`) and write (`mut T*`) borrows obey lexical `{}` block scoping and aliasing rules.

- Dereferencing of `raw T*` pointers occurs only inside `trusted` blocks.

4. **LLVM IR Code Generation:** Once safety checks succeed, ownership and nullability qualifiers have served their purpose and are erased. The verified AST is lowered into LLVM Intermediate Representation (LLVM IR):
   - `own T*`, `mut T*`, `T*?`, and `raw T*` → standard LLVM pointer types
   - `new T` → heap allocation call (`malloc`)
   - `free(x)` → memory release call (`free`)
   - `defer` statements → inserted automatically at the exit points of their enclosing `{}` block
5. **Linking & Binary Generation:** Finally, the generated LLVM IR is linked with required C libraries and compiled into a single native executable binary.

7.2 C Library Integration

SafeC supports interoperability with existing C libraries (such as standard I/O or system utilities) at the LLVM IR stage:

- Because SafeC erases its ownership annotations after compile-time checking and uses standard C data layouts, its generated LLVM IR is directly compatible with compiled C code.
- External C libraries are compiled into LLVM IR and linked alongside the SafeC module:
  - SafeC Source (`.sc`) → SafeC Compiler → SafeC LLVM IR
  - C Library Source (`.c`) → C Frontend → C Library LLVM IR
  - Combined LLVM IR → Linker & Backend → Native Executable
- Calls to unchecked external C functions that manipulate raw memory can be wrapped inside `trusted` blocks using raw pointers, allowing SafeC programs to reuse existing C ecosystems while keeping the rest of the program statically checked.